

Production deployment

AI Architect



Проверить, идет ли запись

Меня хорошо видно & слышно?





Степанов Николай

Data Science Lead

- 6 лет опыта в ML с разными задачами — от разработки моделей до общения бизнесом и управления командой
- 3 публикации на международных конференциях про ML в телекоме
- Работаю в Emerging Travel Group

 @nikolaistepanov

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе #OTUS
AI-Architect-2025-12



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ



ФИНИШ



Маршрут вебинара

Знакомство

Цели и смысл

CI/CD

Сценарии вывода в production

Варианты архитектур DS сервисов

Основные тезисы



Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

Назвать разницу в CI/CD для классической разработки и ML

Чтобы релизы моделей были управляемыми

Выбрать стратегию релиза модели и трафик-контроль

Чтобы снижать риск при релизе в production

Выбрать паттерн деплоя инференса

Чтобы не переплатить за инфраструктуру и не переделывать систему, когда нагрузка вырастет



CI/CD

CI/CD — это

подход непрерывной интеграции и непрерывной доставки/развертывание, направленный на оптимизацию и ускорение жизненного цикла разработки программного обеспечения



Классический CI/CD

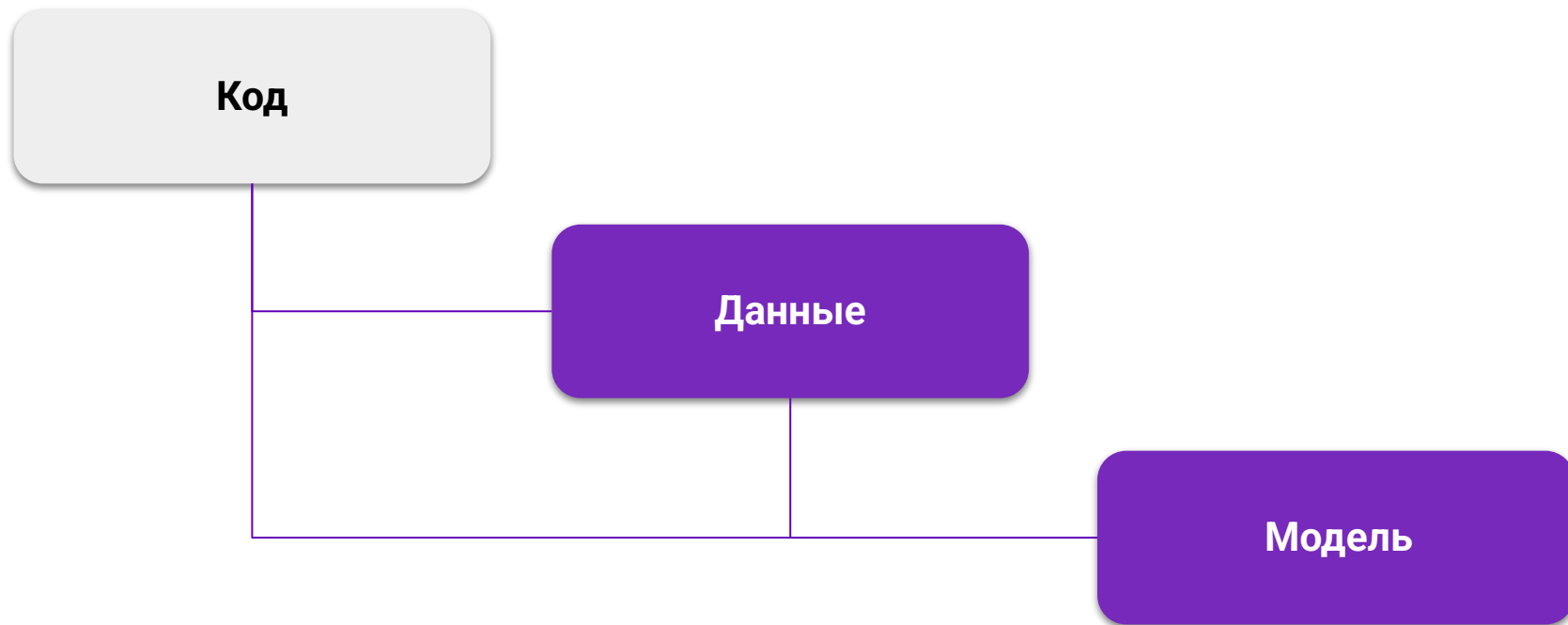


Какие части этой схемы остаются теми же в ML, а какие требуют принципиального расширения?

ML CI/CD

Код

ML CI/CD



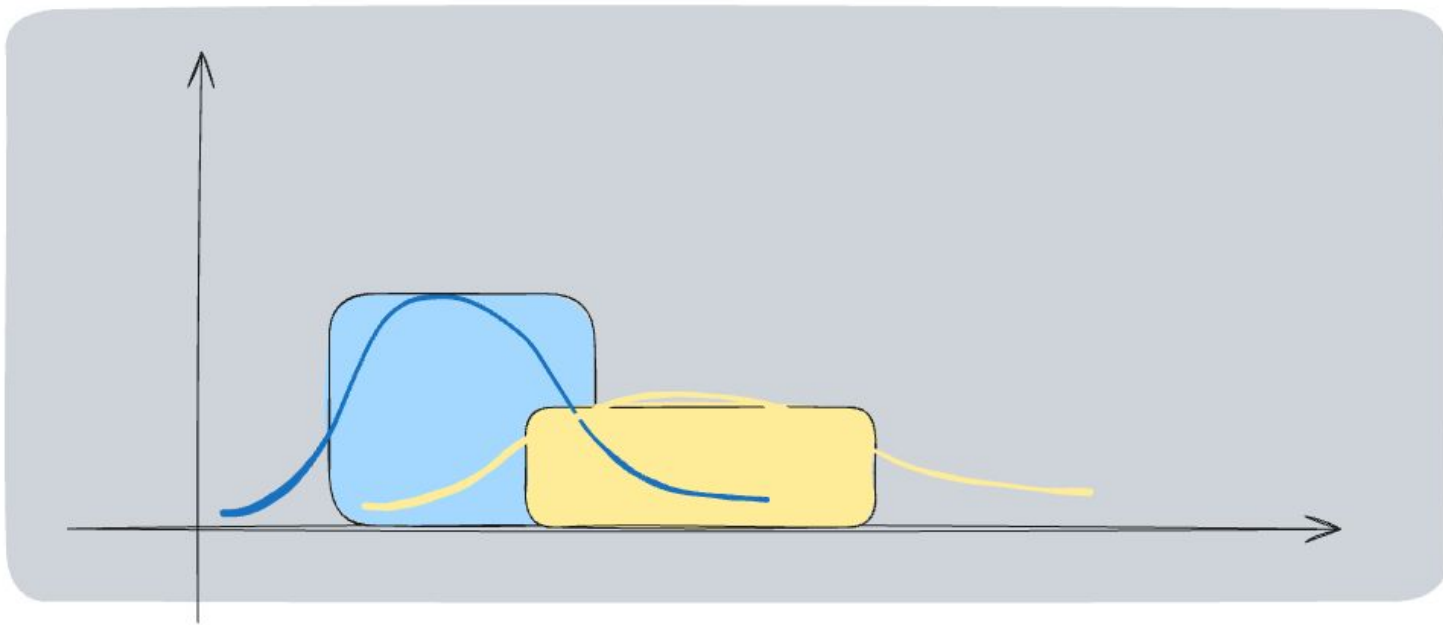
ML CI/CD. Данные

Versioning & Lineage	Data Quality & Contracts	Drift & Monitoring
<i>Dataset snapshots / hashes</i>	<i>Schema + constraints + nulls/ranges</i>	<i>Training-serving skew</i>
<i>Data lineage: raw → features → train/val/test</i>	<i>Data contracts (producer ↔ consumer)</i>	<i>Data drift / concept drift signals</i>
<i>Tools: DVC, lakeFS, Delta/Iceberg/Hudi, MLflow metadata</i>	<i>Tools: Great Expectations, Pandera, dbt tests, Soda</i>	<i>Tools: Evidently, WhyLabs, Arize, + Prometheus/Grafana</i>

ML CI/CD. Модель

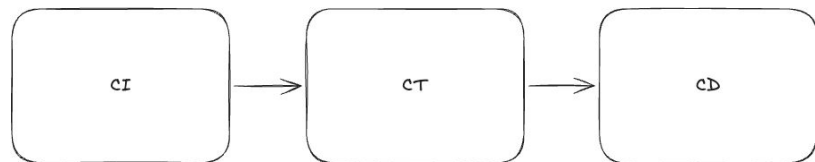
Repo & Structure	Build & Dependencies	Tests & Quality Gates (code + model)
<i>Monorepo vs multi repo (train / serve / features)</i>	<i>Reproducible env: pinned deps + lockfiles</i>	<i>Software tests: unit + integration (transforms, API)</i>
<i>Packaging: libs + pipelines + serving</i>	<i>Containers as artifacts (train/serve images)</i>	<i>Model quality gates: metric thresholds + regression vs baseline</i>
<i>CI hygiene: code review, pre-commit, conventions</i>	<i>Hermetic builds (build cache, deterministic images)</i>	<i>Slicing & risk gates: segments, fairness, robustness, calibration</i>

ML CI/CD. Модель. Пример

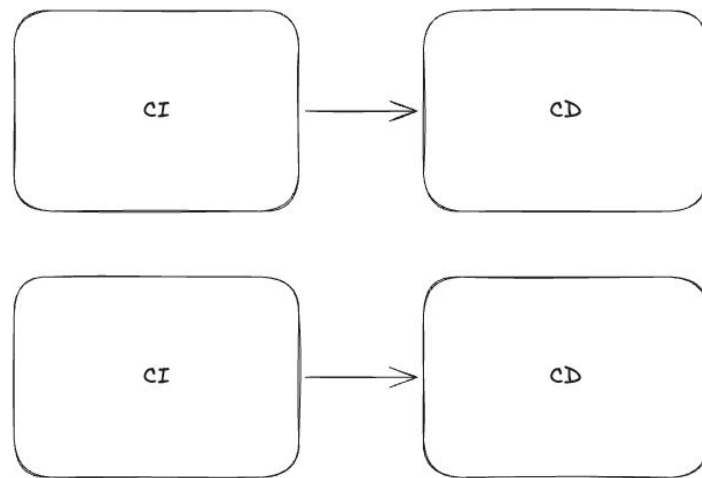


Есть два типа людей

Объединяй и царствуй



Разделяй и властвуй



Вопросы



если есть вопросы



если вопросов нет

Ключевые тезисы блока

1. В классическом CI/CD главный артефакт — код; в ML — код + данные + модель
2. Тесты в ML — это не только pass/fail кода
3. Основа ML CI/CD — воспроизводимость и трассируемость



Сценарии вывода в Production

Какие типы релизов вы можете назвать?



Blue/Green и Rolling release

- Blue/Green: два окружения, switch трафика рубильником
- Rolling: постепенная замена инстансов в одном окружении
- Rollback: быстрый (blue/green) / зависит от стратегии (rolling)
- Хорошо для: сервисов с предсказуемым поведением

Canary release

- Новая версия получает X% трафика, доля растёт по шагам
- Метрики-гейты: SLO (latency/errors) + KPI/ML метрики
- Rollback: 0% на canary за минуты
- Поддержка: ingress/service mesh/traffic router



Canary release. K8S. Ingress

```
# canary-ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: my-model-ingress-canary
```

```
  annotations:
```

```
    kubernetes.io/ingress.class: "nginx"
```

```
    nginx.ingress.kubernetes.io/canary: "true"
```

```
    nginx.ingress.kubernetes.io/canary-weight: "5" #
```

```
Направить 5% трафика
```

```
spec:
```

```
  rules:
```

```
  - host: mymodel.example.com
```

```
    http:
```

```
      paths:
```

```
      - path: /
```

```
        pathType: Prefix
```

```
        backend:
```

```
          service:
```

```
            name:
```

```
my-model-canary-svc # Трафик для
```

```
канарейки идет сюда
```

```
port:
```

```
  number: 80
```



Canary release. K8S. Istio

```
# canary-istio.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: my-model-virtual-service
spec:
  hosts:
    - "mymodel.example.com"
  http:
    - route:
        - destination:
            host: my-model-stable-svc
            weight: 95 # 95% трафика
        - destination:
            host: my-model-canary-svc
            weight: 5 # 5% трафика
```



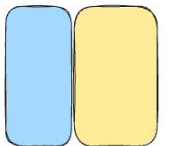
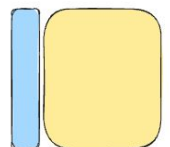
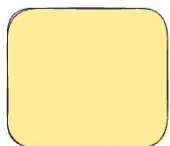
Shadow / Dark launch

- Запросы дублируются на новую версию, но ответ пользователю дает стабильная версия
- Проверяем: ошибки, latency, совместимость, распределение предсказаний
- Нулевой риск для пользователей
- Ограничение: не измеряет бизнес-эффект напрямую

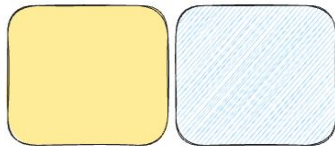
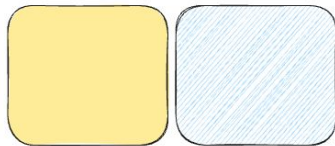
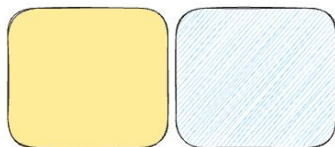
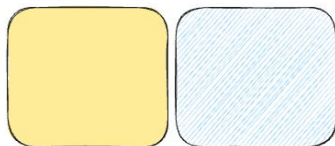


Сравнение

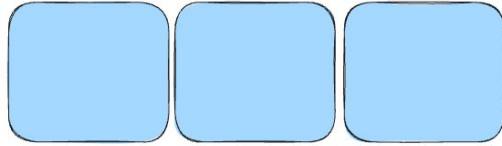
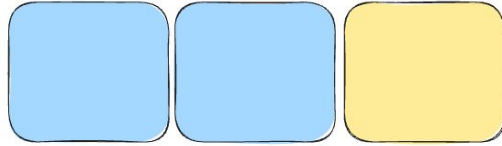
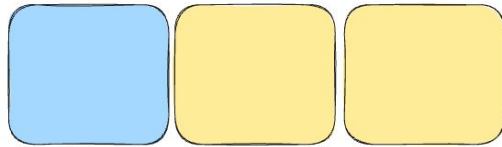
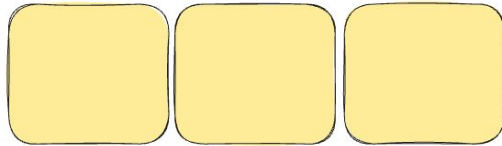
1



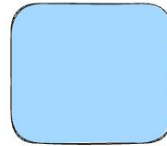
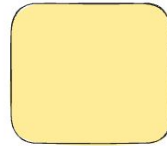
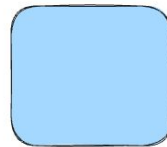
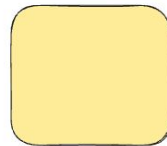
2



3

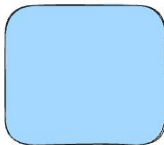
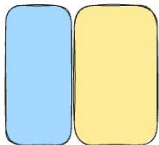
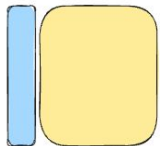
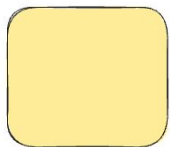


4

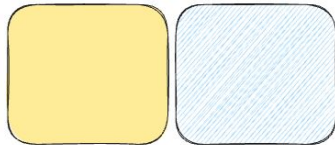
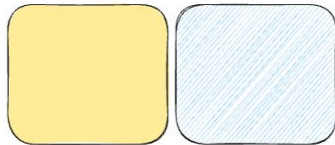
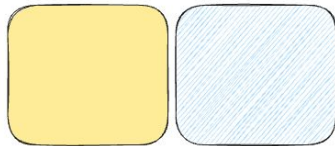
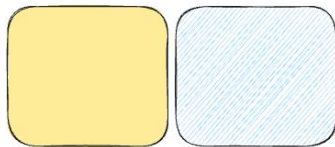


Сравнение

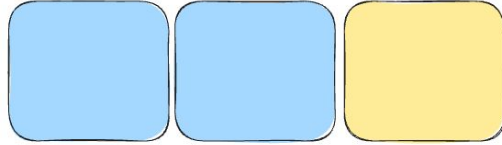
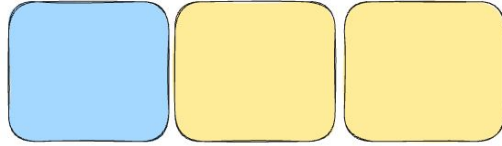
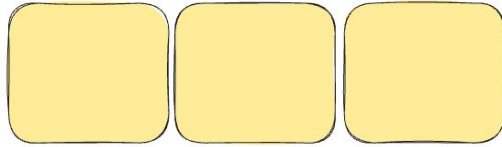
Canary



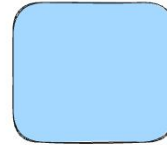
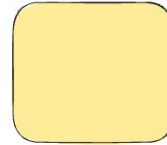
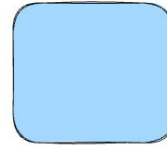
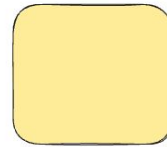
Shadow



Rollout



Blue/Green



Model routing

- A/B test: сравнение двух моделей по KPI (через canary/flags)
- Champion/Challenger: stable vs candidate в registry + промоут
- Shadow → Canary → A/B → Promote
- Online learning / bandits (если применимо): динамическое перераспределение трафика



Model routing

The screenshot shows the mlflow 2.10.0 interface. The 'Models' tab is active, displaying the 'mnist_model_prod' model. A modal titled 'Add/Edit alias for model version 8' is open, showing a list of aliases: '@ challenger' (selected), '@ new_alias', and 'champion'. The modal also includes a 'validation_status' field and a 'Description' field. The background shows a table of model versions (3-8) with columns for Version, Registered at, validation_status, and Description.

mlflow 2.10.0 Experiments Models

Registered Models > **mnist_model_prod**

Created Time: 2023-09-25 12:10:09 Last Modified: 2024-01-19 10:35:06

> Description Edit

> Tags

> Versions Compare

Add/Edit alias for model version 8

Aliases allow you to assign a mutable, named reference to a particular model version. [Learn more](#)

@ challenger @ new_alias

champion This version ✓

champion Version 7

new_alias ✓

validation_status: pending

@ challenger

validation_status: approved

@ champion

Add

Add

Add

Add

Add

Add

New model registry UI

Version	Registered at	validation_status	Description
Version 8	2023-09-25 12:38:44	validation_status: pending	Uses decision tree
Version 7	2023-09-25 12:38:41	validation_status: approved	Uses a CNN based approach
Version 6	2023-09-25 12:38:38	validation_status: approved	
Version 5	2023-09-25 12:38:35		
Version 4	2023-09-25 12:38:32		
Version 3	2023-09-25 12:38:30		

Feature flags

Deploy $\neq$ Release: код уже в production, включаем поведение правилами

Feature flag — условие в коде, которое включает/выключает поведение без нового деплоя. Позволяет отделить deploy (выложили код) от release (включили для пользователей).

Как используется:

- Progressive rollout: 1% $\rightarrow$ 5% $\rightarrow$ 25% $\rightarrow$ 100%
- Сегментация: geo / tenant / cohort / internal users
- Kill switch: быстро выключить при деградации SLO/KPI
- Основа для A/B: контрольная и тестовая группы



Feature flags. Custom

```
def choose_model(user_id: str) -> str:
    # детерминированный бакетинг (пример)
    bucket = hash(user_id) % 100
    return "model_v2" if bucket < 10 else "model_v1"    # 10% трафика на v2

def predict(request, user_id):
    model_name = choose_model(user_id)
    try:
        return models[model_name].predict(request)
    except TimeoutError:
        return models["model_v1"].predict(request)    # fail-safe fallback
```



Feature flags. Unleash

```
from UnleashClient.constants import UnleashContextFields

def choose_model(user_id: str, tenant: str, country: str) -> str:
    # Контекст нужен для:
    # - userId: gradual rollout (percentage)
    # - properties: сегментация по tenant/country и т.п.
    context = {UnleashContextFields.USER_ID: user_id, UnleashContextFields.PROPERTIES:
{"tenant": tenant, "country": country}}
    flag_name = "new_model_v2"
    # Unleash решит:1) включен ли флаг 2) попадает ли user_id в процентный rollout 3)
проходит ли сегментацию по properties
    enabled = client.is_enabled(flag_name, context=context, default_value= False)

    return "model_v2" if enabled else "model_v1"
```



Вопросы



если есть вопросы



если вопросов нет

Ключевые тезисы блока

1. Deploy $\neq$ Release
2. Стратегии релиза отличаются по сигналу / риску / стоимости
3. Решение о выкатке должно опираться на 3 вещи: SLO, бизнес-KPI, ML метрики
4. Качественный релиз невозможен без наблюдаемости и разметки эксперимента



Варианты архитектур DS сервисов

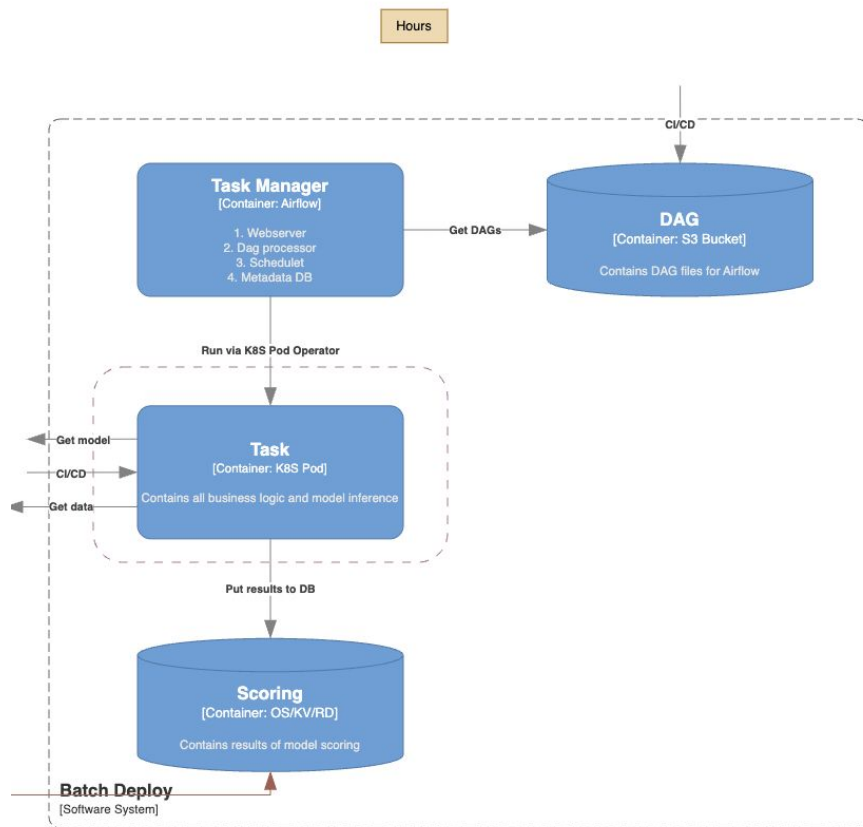
Три сценария

Синхронный (sync)

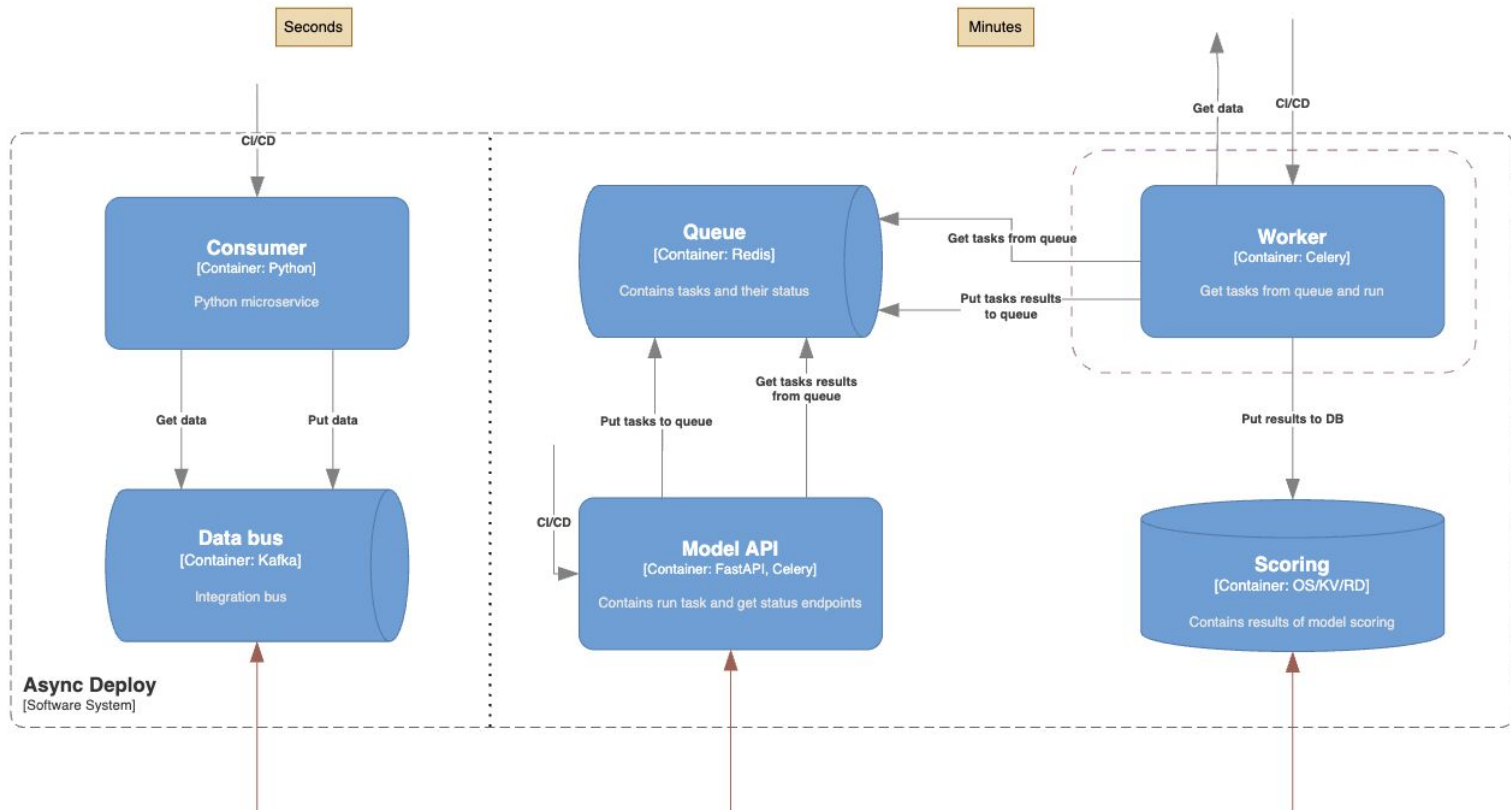
Асинхронный (async)

Пакетный (batch)

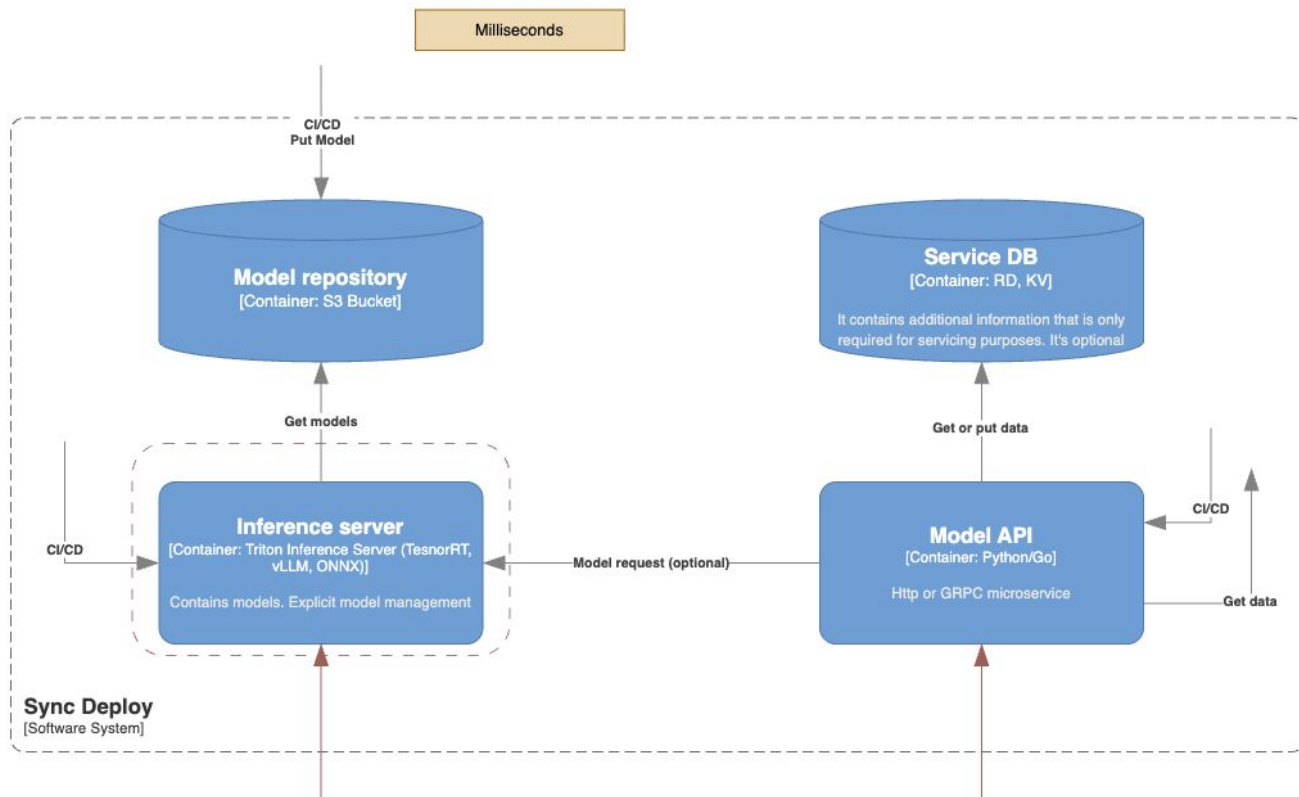
Batch



Async



Sync



Как выбрать?

1. Требуемая задержка: ms / sec—min / hours-days
2. Критичность для пользователя: блокирует UX или нет
3. Нагрузка и стоимость: QPS, пики, GPU/CPU
4. Свежесть результата: нужен сейчас или достаточно обновлять раз в день/месяц
5. Надежность: retries, backpressure, деградация
6. Где живут фичи: online store/cache vs DWH snapshot



Вопросы



если есть вопросы



если вопросов нет

Кейс А — Risk pricing в страховании

Каждую ночь пересчитать риск-скор для всех активных полисов
и сформировать витрину для тарификации и отчётности



Кейс В — LLM: генерация ответа в support-чате

Пользователь пишет в чат поддержки. Нужно сформировать draft reply оператору на основе истории диалога и базы знаний



Кейс С — Аномалии в IoT/промышленном оборудовании

Датчики шлют телеметрию. При обнаружении аномалии нужно сразу поднять тревогу и потенциально остановить линию.



Ключевые тезисы блока

1. Паттерн деплоя выбирается требованиями, а не типом модели



**Подведём
итоги занятия**

Ключевые тезисы занятия

1. ML CI/CD = управление тремя артефактами: код + данные + модель
2. Качество в ML — это gates, а не pass/fail
3. Deploy $\neq$ Release
4. Безопасный стандарт для моделей: shadow (техника) $\rightarrow$ canary (контролируемый риск) $\rightarrow$ A/B (доказательство эффекта) $\rightarrow$ promote/rollback
5. Feature flags/model routing дают максимальную управляемость
6. Паттерн деплоя выбирается требованиями бизнеса



Теперь вы сможете

Назвать разницу в CI/CD для классической разработки и ML

Чтобы релизы моделей были управляемыми

Выбрать стратегию релиза модели и трафик-контроль

Чтобы снижать риск при релизе в production

Выбрать паттерн деплоя инференса

Чтобы не переплатить за инфраструктуру и не переделывать систему, когда нагрузка вырастет



Список материалов для изучения

1. Статья: [Continuous Training](#)
2. Статья: [Древние свитки CI/CD](#)
3. Документация: [Unleash](#)
4. Документация: [Istio Canary Deploy](#)
5. Документация: [Model Registry Workflows](#)



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующие вебинары по проектной работе



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

**You are our
OTUS heroes**

